Software design — a TL;DR

UML notation and the four levels of design, mapped onto a Flutter · Clean Architecture feature.

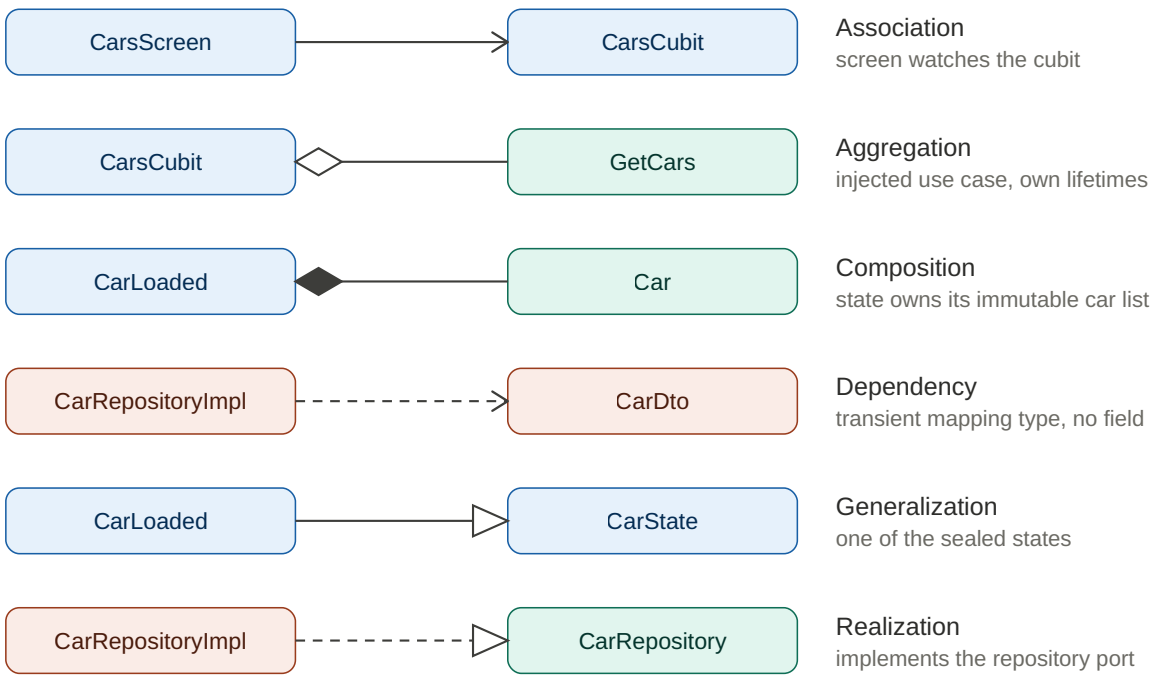
Reference sheet · the Cars vertical (CatsCarsCoins) as the running example

Every design principle below is a lever on the same two quantities: **coupling** and **cohesion**. SOLID, the component principles, and Clean Architecture are just those two ideas applied at increasing scope. This sheet is the map.

UML in one minute

UML (Unified Modeling Language) is a standardized visual notation for the structure and behavior of object-oriented systems. It splits into **structural** diagrams (the static shape: class, component, package, deployment) and **behavioral** diagrams (how it runs: use case, sequence, activity, state machine). For day-to-day BLoC / Clean Architecture work, three earn their keep: the **class** diagram, the **sequence** diagram, and the **state machine** (which maps almost 1:1 onto a sealed state hierarchy).

The part worth memorizing is the class-diagram relationship notation — the head shape tells you the relationship. Here are all six, landed on real types from the Cars vertical:



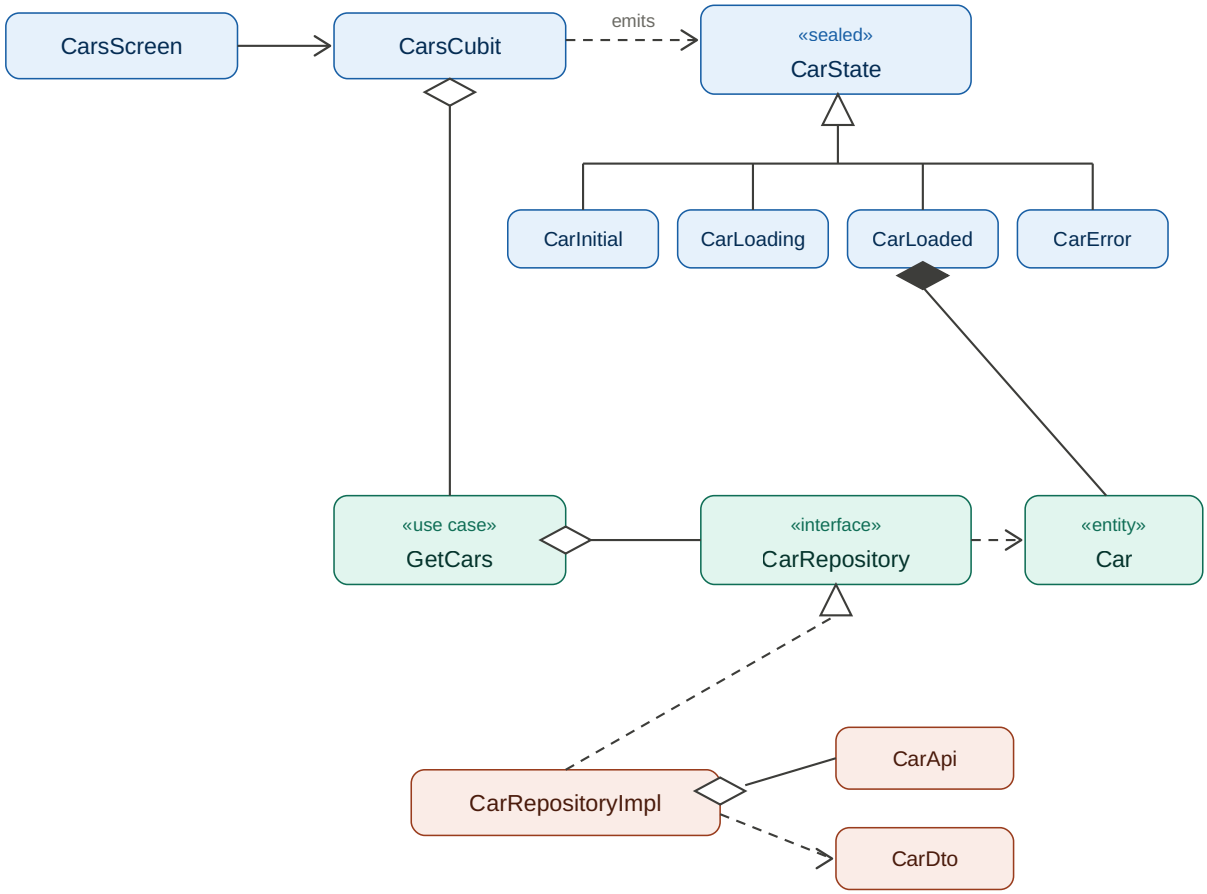
Box color = layer: blue = presentation · teal = domain · coral = data

The six UML class relationships. Diamonds sit on the owning ("whole") end; triangles point at the parent / interface.

Aggregation vs composition is the pair most often confused. The **filled** diamond (composition) means the part dies with the whole — `CarLoaded` owns its immutable car list. The **hollow** diamond (aggregation)

means shared, independent lifetime — a Koin-injected collaborator the holder didn't construct. In a DI-wired app, most collaborators are aggregation.

Wired together, the whole Cars vertical looks like this — the same six relationships, now shown as one connected class diagram across the three Clean Architecture tiers:

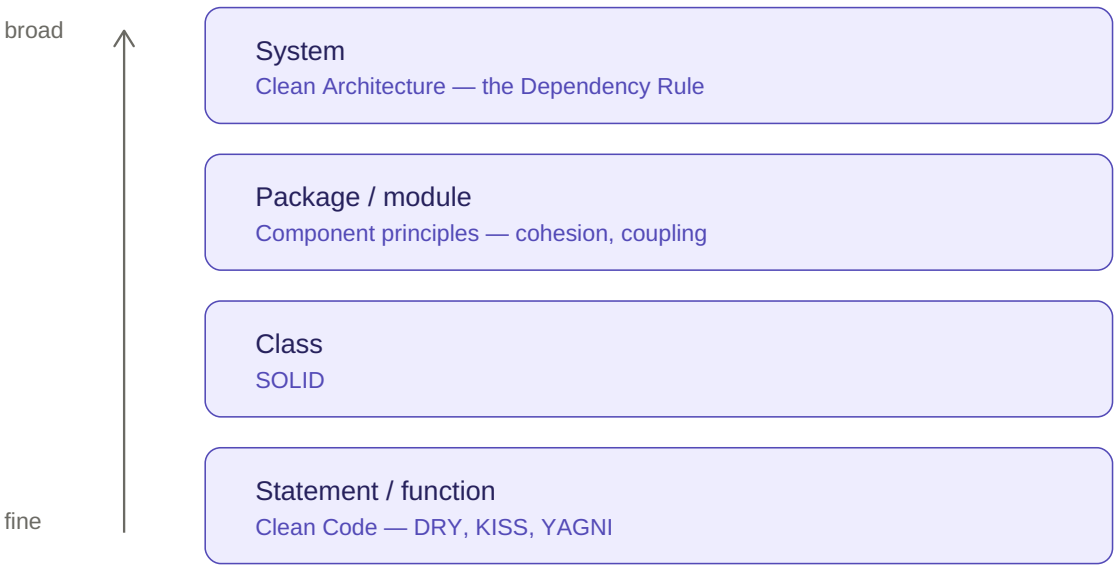


Box color = layer: blue = presentation · teal = domain · coral = data

The Cars vertical as a class diagram. Dependencies flow downward and inward: presentation depends on domain abstractions; the data layer realizes them.

The four levels of design

Design principles form a hierarchy by scale. Each level has its own named principles, and they compose upward — clean functions make clean classes possible, clean classes make clean components possible, and so on.



From the finest grain (a line of code) to the broadest (system architecture). Higher = wider scope.

1 · Statement / function — Clean Code

Intention-revealing names, small functions that do one thing at one level of abstraction, few arguments, no hidden side effects, command–query separation. This is where `DRY`, `KISS`, and `YAGNI` bite hardest.

2 · Class — SOLID

Principle	In one line
S — Single Responsibility	One reason to change (one actor).
O — Open / Closed	Extend behavior without modifying existing code — polymorphism, not if-ladders.
L — Liskov Substitution	Subtypes are usable anywhere the base type is, without surprises.
I — Interface Segregation	Many focused interfaces beat one fat one.
D — Dependency Inversion	Depend on abstractions; policy doesn't know about detail.

3 · Package / module — Component principles

The level most often skipped — and what actually keeps a monorepo sane. Two clusters:

Cohesion — what belongs together

REP — reuse / release equivalence: the unit of reuse is the unit of release.

CCP — common closure: things that change together ship together.

Coupling — how units relate

ADP — acyclic dependencies: no cycles in the dependency graph.

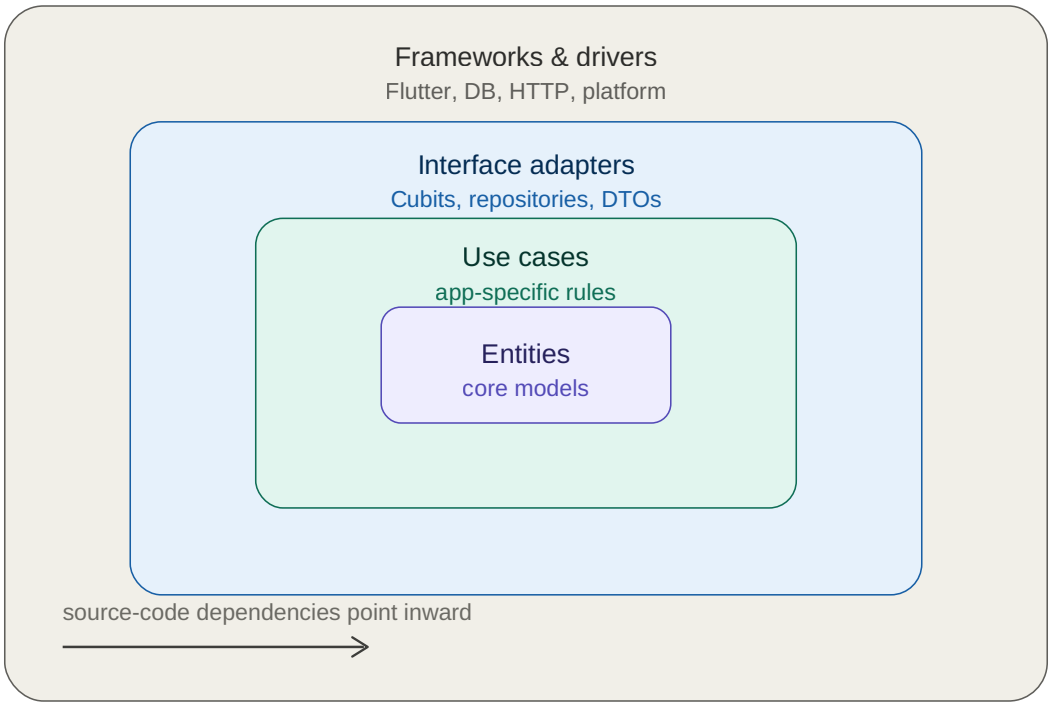
SDP — stable dependencies: depend in the direction of stability.

CRP — common reuse: don't force users to depend on what they don't use.

SAP — stable abstractions: stable components should be abstract.

4 · System — Clean Architecture

Concentric layers governed by one rule. Source-code dependencies point **inward only**: outer, volatile detail depends on inner, stable policy — never the reverse. Boundaries are crossed through abstractions, which is just Dependency Inversion applied architecturally.

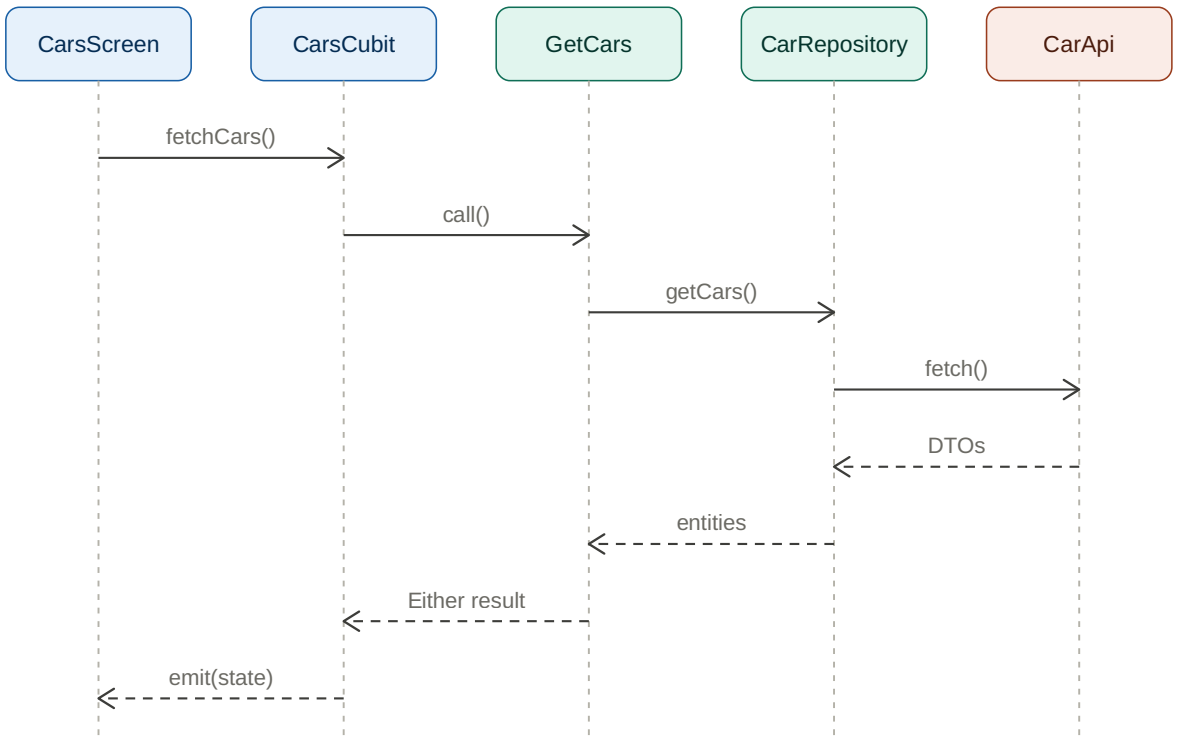


The Dependency Rule: an inner layer knows nothing about the layers outside it.

It all runs on the Dependency Rule

WORKED EXAMPLE — ONE READ THROUGH THE CARS VERTICAL

Watch what the rule looks like in motion. Control flows **inward** (solid arrows) as the tap descends toward the data source; the data returns **outward** (dashed arrows) as DTOs become entities become an emitted state. Yet the whole way down, the *source-code dependency* still points inward — the domain owns `CarRepository` as an abstraction, and `CarApi` in the data layer realizes it.



Solid = control flowing inward. Dashed = data returning outward. Colors are the layers from the diagram above.

> **The through-line.** Read the four levels top to bottom and the same instruction repeats at every scale: raise cohesion, lower coupling, and point dependencies toward the things that change least. Get that right at the function level and the class, component, and system levels tend to fall into place.